

Broken Minimum Spanning Tree

Resolução de Problemas com Grafos — Prof. Me. Ricardo Carubbi

- Gabriel Trajano — 2410361
- Davi Lira Cysne — 2410372
- Thiago Holanda — 2410384
- Lucas Bezerra — 2310237
- Artur da Ponte — 2410368

Linguagem: **Python 3** · Plataforma: **Kattis**

- 📄 **Notas do apresentador (30–45s):** Bom dia a todos. Somos o Grupo H e vamos apresentar a solução para o problema *Broken Minimum Spanning Tree* do Kattis. Este trabalho faz parte do Trabalho Prático 1 da Unidade 3 da disciplina de Grafos. Nosso objetivo é mostrar como modelamos o problema, qual estratégia algorítmica adotamos e por que a solução está correta. Usamos Python 3 e obtivemos aprovação total nos testes do Kattis.

Enunciado e Objetivo

Árvore geradora mínima quebrada

Ethan recebeu a tarefa de encontrar uma árvore geradora mínima de um grafo não direcionado, conexo e ponderado. No entanto, ele interpretou mal a tarefa e encontrou uma árvore geradora que pode não ser mínima. Para tornar sua árvore geradora mínima, você realiza uma sequência de trocas de arestas. Uma troca de aresta remove uma aresta da árvore geradora e adiciona uma aresta do grafo que não está atualmente na árvore geradora. Após cada troca de aresta, a árvore deve continuar sendo uma árvore geradora. Qual é o número mínimo de trocas de arestas que você deve realizar para corrigir a árvore geradora mínima de Ethan?

Entrada

A primeira linha da entrada contém dois números inteiros n ($2 \leq n \leq 2\,000$) e m ($n - 1 \leq m \leq 3\,000$), onde n é o número de nós no grafo e m é o número de arestas no grafo. Os nós são numerados de 1 para n .

Cada um dos próximos m linhas contém três números inteiros u, v ($1 \leq u, v \leq n, u \neq v$), e w ($1 \leq w \leq 10^9$), representando uma aresta que conecta nós u e v com peso w . As bordas são numeradas a partir de 1 para m .

É garantido que o grafo é conexo. O primeiro $n - 1$ arestas da entrada são a árvore geradora inicial de Ethan. O grafo pode não ser simples; pode haver múltiplas arestas entre o mesmo par de nós.

Saída

Imprima um único número inteiro k , que é o número mínimo de trocas de arestas necessárias para tornar a árvore geradora uma árvore geradora mínima. Em seguida, a saída é k linhas, cada uma com dois números inteiros a e b , onde a é o número da aresta a ser removida e b é o número da aresta a ser adicionada. Se houver vários conjuntos de k trocas de arestas que funcionem, qualquer uma será aceita.

O problema

Ethan forneceu uma **árvore geradora** que pode não ser mínima. Precisamos transformá-la em uma **MST** com o **mínimo número de trocas de arestas**.

Operação de troca

Remove-se 1 aresta da árvore e adiciona-se 1 aresta externa, **mantendo a estrutura de árvore geradora**.

i Saída esperada: quantidade mínima de swaps + lista de pares (**remove_id**, **add_id**).

Notas do apresentador (30–45s): Em termos simples, o problema nos dá uma árvore geradora qualquer de um grafo ponderado e pede para convertê-la em uma árvore geradora mínima. A única operação permitida é trocar uma aresta da árvore por uma aresta que está fora dela, sempre mantendo a propriedade de árvore geradora. O objetivo é fazer isso com o menor número possível de trocas. A saída do programa deve listar exatamente quais arestas remover e quais adicionar.

Modelagem como Grafo Ponderado

Estrutura do grafo

→ Vértices

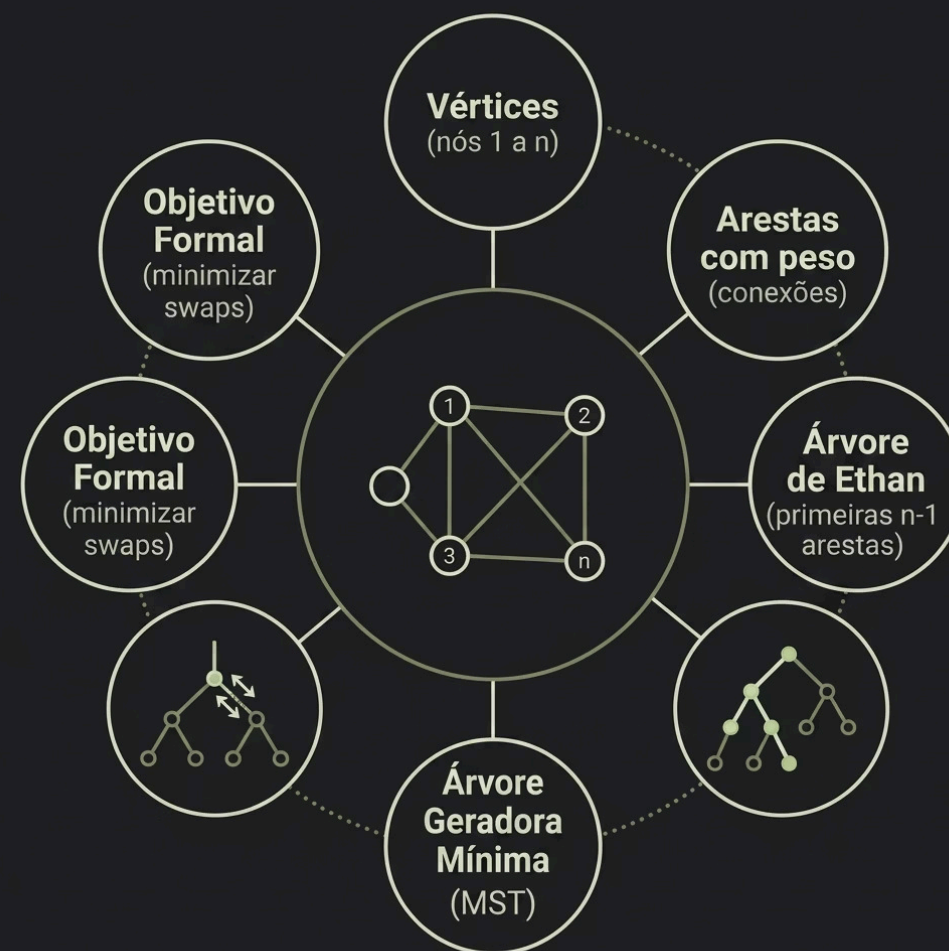
Nós do grafo, numerados de 1 a n

→ Arestas

Conexões com peso; primeiras $n-1$ formam a árvore inicial de Ethan

→ Multigrafo

Pode haver múltiplas arestas entre os mesmos vértices



- ❏ **Notas do apresentador (30–45s):** Modelamos o problema como um grafo ponderado não direcionado. Os vértices são os nós do grafo, e as arestas têm pesos associados. As primeiras $n-1$ arestas da entrada formam a árvore inicial fornecida por Ethan. É importante notar que o grafo pode ser um multigrafo, ou seja, pode existir mais de uma aresta conectando o mesmo par de vértices com pesos diferentes. O objetivo formal é minimizar o número de swaps necessários para chegar a uma MST válida.

Ideia Central da Solução

Etapa 1

Encontrar uma **MST** que preserve o **máximo de arestas** da árvore inicial de Ethan

Etapa 2

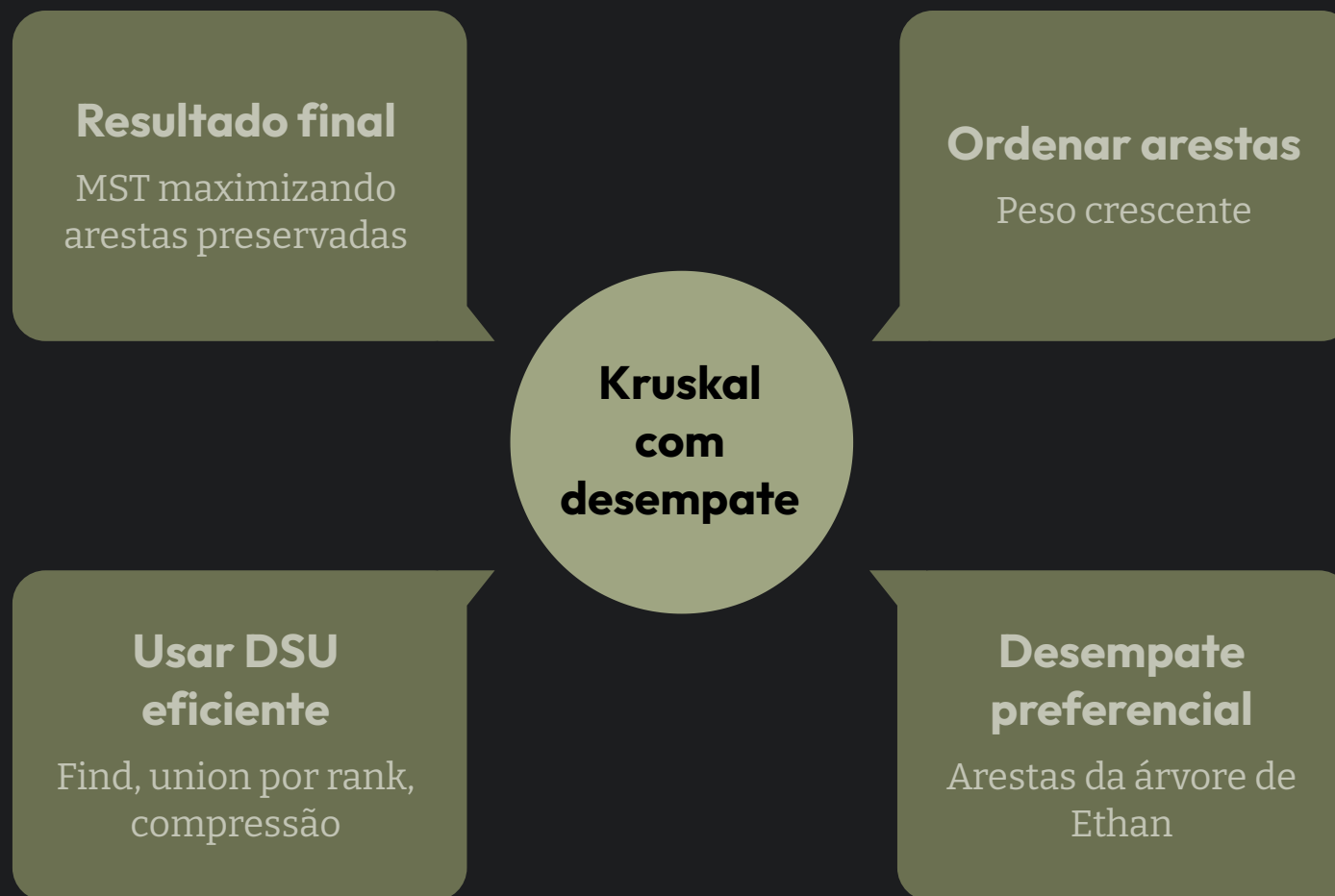
Transformar a árvore de Ethan nessa MST por meio de **swaps válidos**

Insight Principal

Minimizar swaps = maximizar a interseção entre a árvore inicial e uma **MST**

- ❏ **Notas do apresentador (30–45s):** A ideia central é muito elegante: em vez de simplesmente calcular qualquer MST, nós calculamos uma MST que tenha o maior número possível de arestas em comum com a árvore inicial de Ethan. Se maximizamos essa interseção, automaticamente minimizamos o número de trocas necessárias. Isso transforma o problema em uma variação do algoritmo de Kruskal com um critério de desempate inteligente, que veremos no próximo slide.

Algoritmo: Kruskal com Desempate + DSU



O **critério de desempate** garante que, entre arestas de mesmo peso, as que já pertencem à árvore inicial sejam escolhidas primeiro — maximizando a interseção e reduzindo trocas.

- ❏ **Notas do apresentador (30–45s):** Usamos o algoritmo de Kruskal com uma modificação importante. Primeiro ordenamos todas as arestas por peso crescente. Quando há empate de peso, priorizamos as arestas que já estão na árvore de Ethan. Usamos a estrutura Union-Find com path compression e union by rank para detectar ciclos eficientemente. Esse critério de desempate é o que garante que a MST resultante preserve o máximo possível de arestas originais, minimizando assim as trocas necessárias.

Gerando as Trocas (Swap Operations)

Identificação das arestas

to_remove

Arestas da árvore inicial que **não** ficaram na MST

to_add

Arestas que entram na MST e **não** estavam na árvore inicial

Procedimento de swap

Para cada aresta a adicionar:

1. Localizar o caminho entre os vértices na árvore atual
2. Remover a aresta de **maior peso** nesse caminho (marcada em to_remove)
3. Registrar o par (**remove_id**, **add_id**)

Número mínimo de trocas: $|\text{to_remove}| = |\text{to_add}|$

- ❏ **Notas do apresentador (30–45s):** Após construir a MST, identificamos duas listas: to_remove, com arestas da árvore inicial que não ficaram na MST, e to_add, com arestas novas. Para cada aresta que precisamos adicionar, localizamos o caminho entre seus vértices na árvore atual e removemos a aresta de maior peso nesse caminho que esteja marcada para remoção. Registramos cada par remove_id e add_id. O número mínimo de trocas é exatamente o tamanho dessas listas.

Corretude, Complexidade e Casos Especiais

Corretude

- **Kruskal** garante minimalidade do custo total
- Desempate favorece arestas da árvore inicial, reduzindo swaps
- Cada swap preserva a estrutura de **árvore geradora**

Casos Especiais

- Pesos iguais entre arestas distintas
- Multigrafo com arestas paralelas
- Árvore inicial já é **MST** → 0 trocas
- Cenário com muitas trocas necessárias

Complexidade

$O(m \log m)$

Ordenação das arestas

$O(m \alpha(n))$

Kruskal + DSU

$O(n^2)$

Atualizações de caminho (pior caso)

Total: $O(m \log m + n^2)$

- ❏ **Notas do apresentador (30–45s):** Sobre corretude: Kruskal garante que o custo total da árvore é mínimo. O critério de desempate garante que, entre todas as MSTs possíveis, escolhemos aquela com maior interseção com a árvore inicial. Cada swap individualmente preserva a propriedade de árvore geradora. A complexidade total é dominada pela ordenação $O(m \log m)$ e pelas atualizações de caminho que podem chegar a $O(n^2)$ no pior caso. Os casos especiais incluem multigrafos, pesos iguais e o caso trivial onde a árvore já é uma MST.

★ Challenge BETA

🏆 Ranklists

📁 Jobs (5)

</> Languages

i Info

? Help



Search Kattis 🔍

DATE	PROBLEM	JUDGEMENT	RUNTIME	LANGUAGE	TEST CASES
23:32:24	Broken Minimum Spanning Tree	✅ Accepted	0.28 s	Python 3	26/26
🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢					

Submitted Files

Submission history

Files submitted

Mainfile: brokenminimumspanningtree.py

brokenminimumspanningtree.py

1 import sys

2 from collections import defaultdict

3

4

5 def main():

6 data = sys.stdin.read().split()

7 if not data:

8 return

Download brokenminimumspanningtree.py

Resultado Final

26/26

Testes Aprovados

Accepted no Kattis — todos os casos de teste

✅ Solução **correta, eficiente** para os limites propostos e alinhada ao conteúdo de **MST** da disciplina.

Perguntas?

📄 **Notas do apresentador (30–45s):** Como resultado final, nossa solução foi aprovada em todos os 26 testes do Kattis com status Accepted. Isso confirma que a modelagem, o algoritmo e a implementação estão corretos. A solução é eficiente para os limites do problema e está completamente alinhada com o conteúdo de MST estudado na disciplina. Agradecemos a atenção e estamos à disposição para perguntas.